CIS 1904: Haskell

Property-Based Testing

Logistics

- HW 10 due last night
- Last homework, HW11 will be released tomorrow

Property-Based Testing

Write tests for *properties* that should hold of all output.

E.g., the output of take should:

- Be a prefix of the input list
- Have length n, for 0 <= n < length xs
- etc.

We can then *randomly generate* arbitrarily many inputs and check that the property holds for all of the outputs.

How does QuickCheck generate inputs?

The standard library contains a number of prebuilt *generators*.

- E.g. integers, lists of types for which it has generators
- For custom types, we must build our own generators
  - We will learn how to do this in a later lecture

Arbitrary

```
class Arbitrary a where
    arbitrary :: Gen a
    shrink  :: a -> [a]
```

Arbitrary

```
class Arbitrary a where
    arbitrary :: Gen a
    shrink  :: a -> [a]
```

What is a generator for type `a`?

Gen

Under the hood:

```
newtype Gen a = MkGen
    {
    -- Run the generator on a particular seed and size.
    unGen :: QCGen -> Int -> a
    }
```

Gen

Under the hood:

```
newtype Gen a = MkGen
    {
    -- Run the generator on a particular seed and size.
    unGen :: QCGen -> Int -> a
    }
```

`Gen` is a monad!

- `return x` is the generator that always produces `x`
- `g1 >>= (\x -> g2)` sequences generation of values

Gen

In a moment, we will write an `Arbitrary` instance for `Pair a b`, assuming `a` and `b` are instances of `Arbitrary`.

Run `sample (arbitrary :: Gen (Pair Int Bool))` to get a sense of the output.

QuickCheck

What would the output of `sample (arbitrary :: Gen (Pair a b))` have been?

```
sample (arbitrary :: (Arbitrary a, Arbitrary b) => Gen (Pair a b))
→   Pair () (), Pair () (), Pair () (), Pair () (), Pair () ()
```

If QuickCheck is left to pick a value for `a`, it will usually pick `()`.

QuickCheck

What would the output of `sample (arbitrary :: Gen (Pair a b))` have been?

```
sample (arbitrary :: (Arbitrary a, Arbitrary b) => Gen (Pair a b))
→   Pair () (), Pair () (), Pair () (), Pair () (), Pair () ()
```

If QuickCheck is left to pick a value for `a`, it will usually pick `()`.

```
prop_Foo :: Pair a b -> Property
prop_Foo = …
```

`quickCheck prop_Foo` will only test `Pair () ()`.

Gen

Write an `Arbitrary` instance for `Pair a b`, assuming `a` and `b` are instances of `Arbitrary`. (Ignore `shrink` for now).

Run `sample (arbitrary :: Gen (Pair Int Bool))` to get a sense of the output.

Arbitrary

Standard library combinators:

- `elements :: [a] -> Gen a`
  - given a list of values of type `a`, generate one of those values, with equal probability for each element
  - will crash if input is nonempty

```
elements [1,2,3]
→   3, 1, 2, 2, 3, 2, 1, 1, 3, 1
```

Arbitrary

Standard library combinators:

- `elements :: [a] -> Gen a`
  - given a list of values of type `a`, generate one of those values, with equal probability for each element
  - will crash if input is nonempty

Write an `Arbitrary` instance for `CardinalDirection`. (Ignore `shrink`.)
Run `sample (arbitrary :: Gen (Tree Int))` to get a sense of the output.

Arbitrary

Standard library combinators:

- `oneof :: [Gen a] -> Gen a`
  - given a list of generators, randomly pick a generator and use it to generate a value, with equal probability for each generator
  - will crash if input is nonempty

```
oneof [return 1, arbitrary]
→   1, 506, -3, 1, 1, -97814, 37, 1, -914, 1
```

Arbitrary

Standard library combinators:

- `oneof :: [Gen a] -> Gen a`
  - given a list of generators, randomly pick a generator and use it to generate a value, with equal probability for each generator
  - will crash if input is nonempty

Write an `Arbitrary` instance for `Tree a`, assuming `a` is an instance of `Arbitrary`. (Ignore `shrink`.)

Arbitrary

Standard library combinators:

- `frequency :: [(Int, Gen a)] -> Gen a`
  - generalization of `oneof`
  - given a list of generators paired with weights, randomly pick a generator and use it to generate a value, with probabilities given by weights
  - will crash if input is nonempty

```
frequency [(4, return 1), (1,arbitrary)]
→   1, 1, -5, 1, 1, 333468, 1, 1, 1, 1
```

Arbitrary

Standard library combinators:

- `frequency :: [(Int, Gen a)] -> Gen a`
  - generalization of `oneof`
  - given a list of generators paired with weights, randomly pick a generator and use it to generate a value, with probabilities given by weights
  - will crash if input is nonempty

Improve the `Arbitrary` instance for `Tree a`.

Arbitrary

Standard library combinators:

- `sized :: (Int -> Gen a) -> Gen a`
  - given a size-limited generator, use that generator with the current size to generate a value
  - `quickCheck` default *maximum* size is 100
  - `quickCheck` will sample a given generator with increasingly large sizes up to the maximum

Arbitrary

Standard library combinators:

```
sized genListAux
  where
    genListAux n = if n <= 0 then return [] else do
        x <- (arbitrary :: Gen Int)
        xs <- genListAux (n - 1)
        return $ x : xs
```

```
→ [], [-87, 4], [7,0,-98341728, -774], [976, -31, -2, 1008, 23, -1], …
```

Arbitrary

Standard library combinators:

- `sized :: (Int -> Gen a) -> Gen a`
  - given a size-limited generator, use that generator with the current size to generate a value
  - default size is 100
  - size can be adjusted with `resize` or `scale`
  - current size can be checked within the `Gen` monad using `getSize`

Improve the `Arbitrary` instance for `Tree a`.

Arbitrary

```
class Arbitrary a where
    arbitrary :: Gen a
    shrink  :: a -> [a]
```

Shrinking

```
prop_Foo :: [Int] -> Property
prop_Foo xs = myFoo xs === referenceFoo xs
```

Shrinking

```
prop_Foo :: [Int] -> Property
prop_Foo xs = myFoo xs === referenceFoo xs
```

```
ghci> quickCheck prop_Foo
*** Failed! Falsifiable (after 4 tests):
[478, 1, 207091, -43, 5197, -202, -43000, 7, -3819, 113, 2191, -552,
-581, 407, 316890, 2, 304, -334, 725, -87424, 3042299, -9221, 38897, 566,
-80, 0, -85220, -9912471, 306618, -9162, 49329, -85808, -53, 404, 3539,
8, -73008, -99863325, 68, 71819, 140, -80041, 671, -2, -94, 58]
```

Shrinking

```
prop_Foo :: [Int] -> Property
prop_Foo xs = myFoo xs === referenceFoo xs
```

```
ghci> quickCheck prop_Foo
*** Failed! Falsifiable (after 4 tests and 3 shrinks):
[0,-1]
```

Shrinking

```
prop_Foo :: [Int] -> Property
prop_Foo xs = myFoo xs === referenceFoo xs
```

```
ghci> quickCheck prop_Foo
*** Failed! Falsifiable (after 4 tests and 3 shrinks):
[0,-1]
```

Shrinking is the process of going from a large counterexample to a small one.

Shrinking

`[-5, -3, 4]`

→ `[ [], [-5], [-3], [4], [-5,-3], [-5, 4], [-3,4]]`
Test succeeds on `[]`, fails on `[-5]`

→ `[ [5], [] ]`
Test fails on `[5]`

→ `[ [0], [] ]`
Test fails on `[0]`

→ `[ [] ]`
Test succeeds on `[]`

Final counterexample: `[0]`
(To view the full shrinking process, use `verboseCheck` instead of `quickCheck`.)

Shrinking

```
shrink  :: a -> [a]
```

- `shrink` produces a list of shrunk versions of the input
  - The output of shrink should NOT include the input
  - QuickCheck tries each element of the list *in order*
- The default implementation of `shrink` returns `[]` (i.e., no shrinking occurs)

Shrinking

Shrinking principles*:

1. Shrink a term to any of its immediate subterms.

   You can use `subterms` to do this.

2. Recursively apply `shrink` to all immediate subterms.

   You can use `recursivelyShrink` to do this.

3. Type-specific shrinkings such as replacing a constructor by a simpler constructor.

*Principles taken verbatim from [the docs]

Shrinking

```
shrinkPair :: (Arbitrary a, Arbitrary b) Pair a b -> [Pair a b]
shrinkPair (Pair x y) = do
    x’ <- shrink x
    y’ <- shrink y
    return (Pair x’ y’)
```

What is the problem with this implementation?

Shrinking

```
shrinkPair :: (Arbitrary a, Arbitrary b) Pair a b -> [Pair a b]
shrinkPair (Pair x y) = do
    x’ <- shrink x
    y’ <- shrink y
    return (Pair x’ y’)
```

What is the problem with this implementation?

```
ghci> shrink (Pair 0 2)
[]
```

Shrinking

```
shrinkPair :: (Arbitrary a, Arbitrary b) Pair a b -> [Pair a b]
shrinkPair (Pair x y) = do
    (x’, y’) <- shrink (x, y)
    return (Pair x’ y’)
```

For more details, see the `Arbitrary2` typeclass.

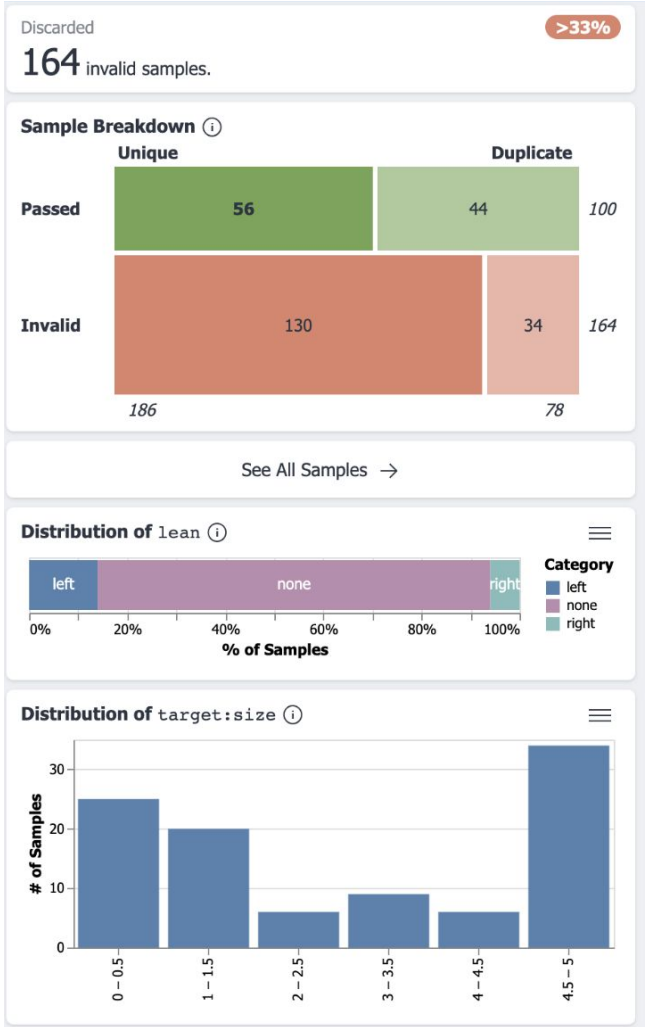
Shrinking

Implement `shrink` for `Tree a`.

Do not use `subterms` or `recursivelyShrink`.

Tyche

- Generator visualization tool
- Fairly new!
- Disclaimer: made by my advisor



Bonus

Exercise: Write a generator for binary search trees.

Invariants:

- All values in the left child of a node are strictly less than the node’s value
- All values in the right child of a node are strictly greater than the node’s value

Hint: The simplest implementation takes in lower and upper bounds for the possible values in the BST.